Restful-booker API Testing: Practice Report and Test Log

Tester: David Remo
Date: Started 28 October 2026, last updated 30 October 2026
Tool: Postman (manual testing first, then automated checks with scripts and the Collection Runner)
System under test: restful-booker, a public practice API at https://restful-booker.herokuapp.com

1. Summary

I tested the restful-booker hotel booking API by hand in Postman. I ran the full lifecycle of a booking (authenticate, create, read, full update, partial update, delete), added negative tests along the way, and logged every result as expected vs actual.

Headline results

- Manual test results: 18 tests run, 12 Pass, 3 Fail and 3 Blocked (definitions in section 3).
- Core create, read, update and delete flows work and return the data that was sent.
- Authorization is enforced: requests with a stale, missing or fake token were rejected with 403.
- Three defects (BUG-001 to BUG-003) and one observation (OBS-001) deviate from standard API conventions (section 5). Restful-booker is a practice API with known quirks, so these are reported as deviations from REST conventions, not as defects in a real product.
- The three Blocked tests (7, 9 and 10) were caused by the shared server resetting its data, not by the API's behavior. I handled this by re-creating my test data and repeating the check (section 6).

2. Scope and approach

Endpoints covered

Endpoint	Method	Purpose
/ping	GET	Health check
/booking	GET	List booking IDs, with optional filters
/booking/{id}	GET	Get one booking
/auth	POST	Create an auth token
/booking	POST	Create a booking
/booking/{id}	PUT	Replace a booking completely
/booking/{id}	PATCH	Change part of a booking
/booking/{id}	DELETE	Remove a booking

Approach: for each endpoint I checked the status code first, then the response body (field names, values and types), then ran negative tests with bad IDs, wrong credentials, missing tokens and incomplete data. I created my own test data immediately before using it, because the server is shared and resets periodically.

3. Test log

Result definitions. Pass: the actual result matched the expected result. Fail: the actual result differed from the expected result, and it is logged as a defect (BUG-nnn). Blocked: the test could not be run properly because of the environment or my test setup, so it says nothing about the API and needs a re-run before drawing conclusions.

Totals: 18 tests: 12 Pass, 3 Fail, 3 Blocked.

Note: the 17 checks in section 10 are automated Postman checks. The totals above are for the 18 manual tests in this log.

#	Request	Expected	Actual	Result
1	GET /ping	201 Created	201 Created (first call took 3.54 s while the server woke up)	Pass (see OBS-001)
2	GET /booking/9999999 (nonexistent ID)	404	404 Not Found	Pass
3	GET /booking/abc (text as ID)	404, not 500	404 Not Found	Pass
4	POST /auth, valid credentials	200 + token	Token returned	Pass
5	POST /auth, wrong password	401	200 OK with body {"reason": "Bad credentials"}	**Fail (BUG-001)**
6	POST /booking, valid data	200 + saved data	200, every field matched what I sent	Pass
7	GET /booking/3456, some minutes after creating it	200 + data	404 Not Found	Blocked (server reset)
8	Re-create booking, then GET it immediately	200 + same data	200, every field matched	Pass
9	PUT /booking/4338 with an old token	200	403 Forbidden	Blocked (stale token)
10	PUT on a booking that no longer	404	405 Method Not	Blocked (see

#	Request	Expected	Actual	Result
	exists, valid token		Allowed	test 18)
11	PUT with valid token on a fresh booking	200 + new data	200, all fields matched the new values	Pass
12	PUT with only firstname in the body	400	400 Bad Request	Pass
13	PATCH with only firstname	200, only firstname changed	200, firstname changed to Dave, all other fields unchanged	Pass
14	DELETE /booking/1518 with valid token	204 No Content	201 Created	**Fail (BUG-003)**
15	PATCH with no token (Cookie header switched off)	403	403 Forbidden	Pass
16	PATCH with a fake token (token=abc)	403	403 Forbidden	Pass
17	GET /booking/275 after it was deleted	404	404 Not Found	Pass
18	DELETE /booking/275 a second time, valid token	404 Not Found	405 Method Not Allowed	**Fail (BUG-002)**

4. Key observations

PUT vs PATCH, proven with real data. The same body, {"firstname": "Dave"}, was rejected by PUT with a 400 (PUT needs every field) and accepted by PATCH, which changed only the first name and kept the other five fields intact.

Create is verified by reading back. A create response that echoes the data does not prove it was saved. Reading the booking back in a separate request did.

Status codes tell you where to look.

- 403: the server does not accept who you are (token problem)
- 404: the thing does not exist
- 405: the server will not do that action on that URL
- 400: the data sent was malformed or incomplete

5. Defects and observations found

How to read this section. A defect is behavior that differs from the expected result. An observation is unusual but harmless behavior worth noting. Severity is how much damage the problem can do; priority is how soon it should be fixed. Environment: the public restful-booker server, tested with Postman between 28 and 30 October 2026.

BUG-001: Failed login returns 200 OK

- **Steps:** send POST /auth with {"username": "admin", "password": "wrong"}
- **Expected:** an error status such as 401 Unauthorized
- **Actual:** 200 OK, with body {"reason": "Bad credentials"}
- **Impact:** clients that rely on the status code will treat a failed login as a success.
- **Severity:** Medium. A client that checks only the status code will treat a failed login as a success.
- **Priority:** Medium. A simple fix that affects every client, but no data is exposed or lost.

BUG-002: PUT or DELETE on a missing booking returns 405 instead of 404

- **Steps:** delete a booking, confirm with a GET that it now returns 404 (test 17), then send DELETE to the same /booking/{id} again with a valid token (test 18).
- **Expected:** 404 Not Found
- **Actual:** 405 Method Not Allowed
- **Impact:** misleading. The method is valid; the resource is missing, so the error does not tell the client what went wrong. Confirmed reproducible in test 18, where a GET had already proved the booking was gone. A similar 405 on PUT in test 10 supports this, but the booking's absence was not confirmed there (most likely a server reset), so test 10 is marked Blocked.
- **Severity:** Low. The wrong code is returned, but no data is changed and the outcome is safe.
- **Priority:** Low. It makes failures harder to diagnose but does not break anything.

BUG-003: DELETE returns 201 Created

- **Steps:** send DELETE to /booking/{id} with a valid token
- **Expected:** 204 No Content
- **Actual:** 201 Created, body "Created"
- **Impact:** 201 means something was created, which is the opposite of a delete. Clients checking for the conventional code would misread the result.
- **Severity:** Low. The booking is deleted correctly; only the status code is unconventional.
- **Priority:** Low. Clients that check for the conventional code will misread the result, but nothing is lost.

OBS-001: Health check returns 201 Created

- **Steps:** send GET /ping.
- **Expected (by convention):** 200 OK.
- **Actual:** 201 Created (test 1). The test passes because 201 is what this API returns for a healthy ping.
- **Impact:** harmless, but a monitoring tool that checks for 200 would report a healthy API as down.

6. Test environment notes

Restful-booker is a shared public server that resets its data and tokens regularly. This caused my three Blocked results (tests 7, 9 and 10):

- A booking read back a few minutes after creation returned 404 (test 7). I re-created it and read it back immediately (test 8), and it worked, so this was a reset and not a defect.

- A saved token stopped working and returned 403 (test 9). A fresh token fixed it.
- A booking vanished between steps, producing a 405 on PUT (test 10). I had not confirmed with a GET that it was gone, so I marked the test Blocked and used test 18 as the confirmed reproduction of BUG-002.

The habit that follows from this: create test data right before using it, and get a fresh token before requests that need one. When a result surprises you, check your own setup before blaming the system.

7. What I learned

- How a request is built: method, URL, headers, body, and how a response is read: status code, body, time.
- The difference between query parameters (after the ?) and URL parameters (the ID in the path).
- Why a list endpoint returns only IDs and a separate endpoint returns the details.
- How token-based authorization works and how to test it with missing and fake tokens.
- How to write a bug report with steps, expected result, actual result and impact.
- How to tell an API defect apart from a test setup problem.
- How to record results honestly: Pass, Fail or Blocked, and why a Blocked test is not a defect.

8. Next steps

- Re-read a booking after a rejected request to confirm its data was not changed.
- Add checks to Get all bookings, which has none yet.
- Automate the negative tests (nonexistent ID, no token, fake token) inside the collection.
- Try the same endpoints with XML bodies to see how the format changes.

9. Status code reference

The first digit tells you who to look at: 2xx means success, 4xx means the request was the problem, 5xx means the server was the problem.

Code	Name	Plain meaning	Where I saw it
200	OK	The request worked and data came back	Get booking, create, PUT, PATCH, and a failed login (BUG-001)
201	Created	Normally: something new was made	/ping (healthy, OBS-001), and DELETE (BUG-003)
400	Bad Request	The data sent was malformed or incomplete	PUT with only firstname
401	Unauthorized	Not logged in, or bad credentials	Not seen; what a failed login should return
403	Forbidden	The server understood but does not accept who you are	Stale, missing and fake tokens (tests 9, 15, 16)

Code	Name	Plain meaning	Where I saw it
404	Not Found	That thing does not exist	Nonexistent ID, text ID, deleted booking
405	Method Not Allowed	That action is not allowed on that URL	PUT or DELETE on a missing booking (BUG-002)
500	Internal Server Error	The server broke	Not seen

10. Automation: Postman scripts and the Collection Runner

After the manual pass, I automated the booking lifecycle with Postman scripts (the Scripts tab, After response) and ran it end to end with the Collection Runner.

What the scripts check

Request	Checks
Ping	Status is 201; responds in under 5 seconds
Get token	Status is 200; a token (text) is returned; the token is saved to a variable
Create booking	Status is 200; bookingid is a number; saved firstname matches what was sent; totalprice is a number; depositpaid is a boolean; the bookingid is saved to a variable
Get one booking	Status is 200; firstname and lastname match the booking that was created
Update booking (PUT)	Status is 200; lastname was replaced with the PUT value; all six booking fields are present
Update booking (PATCH)	Status is 200; firstname changed, and lastname and totalprice were kept
Delete booking	Status is 201 (named as a known deviation, see BUG-003)
Get deleted booking	Status is 404

Get all bookings has no checks yet.

Chaining requests with variables

The Get token and Create booking scripts save the token and the new booking ID. Later requests use them as {{token}} and {{bookingId}}, so the whole chain runs with no copy-pasting. It also survives the server resets described in section 6, because every run creates fresh data and a fresh token.

Result

I ran the collection in order (Get token, Create booking, Get one booking, Update PUT, Update PATCH, Delete, Get deleted booking) in the Collection Runner: **17 tests, 17 passed, 0 failed, 0 errors**, about 7.5 seconds in total with an average response time of 557 ms. A screenshot of the results page is in the repo as runner-results.png.

What I learned from automating

- **A test has to be able to fail.** I changed an expected status on purpose and saw a red FAILED before trusting the test.
- **What you send should differ from the previous state.** The PUT test uses a lastname that Create did not use, so an ignored update would be caught.
- **A test name must match what it checks.** My first PATCH test only checked firstname while being named as if it checked the whole booking, so I extended it to check that other fields were kept.
- **Read the error type first.** One run failed with a ReferenceError ("body is not defined"). That was a mistake in my script, not a defect in the API. An AssertionError means the API returned something different from what the test expected.
- **Document known deviations in the test name.** The delete test expects 201 and says so in its name, so nobody mistakes the API's behavior for correct behavior.